

生命游戏 GUI 程序

项目报告

史云天 赵思涵 万泽奇

2026 年 6 月 27 日

目录

1	程序功能介绍	2
1.1	核心功能概览	2
1.2	运行环境	2
2	项目架构与模块设计	3
2.1	整体架构	3
2.2	核心类设计	3
2.2.1	Cell —— 细胞类	3
2.2.2	Grid —— 网格类	3
2.2.3	Rule —— 规则类	4
2.2.4	Simulator —— 演化引擎	4
2.2.5	Pattern —— 图案类	4
2.3	UI 类设计	4
2.3.1	MainWindow —— 主窗口	4
2.3.2	GridView —— 网格视图	5
2.3.3	SideBar —— 侧边栏	5
2.3.4	规则面板 (RulePanel)	5
2.3.5	图案面板 (PatternPanel)	5
2.3.6	统计面板 (StatsPanel)	5
2.4	对话框类设计	5
2.5	工具类设计	6
3	小组成员分工	6
3.1	协作模式	6
4	项目总结与反思	7
4.1	项目成果	7
4.2	技术收获	7
4.3	不足与改进方向	7
4.4	结语	8

1 程序功能介绍

本项目基于 Qt 框架开发了一款交互式生命游戏（Conway's Game of Life）GUI 程序。生命游戏是由数学家 John Conway 于 1970 年提出的细胞自动机，其核心思想简单而深刻：每个细胞只有“生”和“死”两种状态，但通过简单的局部规则演化，能产生极为复杂的全局行为。

1.1 核心功能概览

1. **细胞绘制与演化**：用户点击网格即可绘制或擦除细胞，点击开始按钮后细胞按照规则自动演化，支持暂停和单步执行，演化速度可调。
2. **规则自定义**：支持经典的 B/S（Birth/Survive）规则系统，用户可自定义出生和存活条件，并提供了多种预设规则：康威生命游戏（B3/S23）、HighLife（B36/S23）、Day & Night（B3678/S34678）、Seeds（B2/S）、Replicator（B1357/S1357）。
3. **邻居类型选择**：支持摩尔邻域（周围 8 格）和冯诺依曼邻域（周围 4 格）两种邻居计算方式。
4. **多状态细胞**：细胞具有年龄属性，每存活一代年龄加一，年龄通过颜色渐变呈现（年轻偏蓝，年老偏红），视觉上直观展示演化历史。
5. **图案系统**：内置多种经典预设图案（滑翔机、滑翔机枪、振荡器、飞船等），用户可从图案面板选取并放置到网格中，也可框选区域保存为自定义图案。
6. **网格模式**：支持固定网格（死边界/环形边界）和无限网格模式，无限网格模式下使用哈希表存储活细胞坐标，内存按需分配。
7. **丰富交互**：支持拖拽平移视图、滚轮缩放、框选区域、右键菜单复制/填充/保存图案。
8. **统计信息**：实时显示当前代数、活细胞数量、运行时间、当前规则等信息。

1.2 运行环境

- 操作系统：Windows 10/11
- 开发框架：Qt 5/6 Widgets
- 编译器：MSVC 或 MinGW（C++17）
- 构建工具：CMake 3.16+

2 项目架构与模块设计

2.1 整体架构

项目采用分层架构设计，核心逻辑层与 UI 层分离。核心层负责游戏逻辑计算，不依赖任何 Qt 图形模块；UI 层负责可视化渲染和用户交互。这种设计使得核心算法易于测试和复用。

模块	职责
core/	核心逻辑层：细胞、网格、规则、演化引擎、图案数据结构
ui/	界面层：主窗口、网格视图、侧边栏、各功能面板
ui/dialogs/	对话框层：网格设置、规则编辑、图案保存
utils/	工具层：图案序列化
data/	数据资源：预设图案库

2.2 核心类设计

2.2.1 Cell ——细胞类

Cell 类

Cell 是生命游戏的最小单元。每个 Cell 拥有两种状态（Dead/Alive）和一个年龄属性。关键方法：

- birth(): 出生，状态设为 Alive，年龄初始化为 1
- kill(): 死亡，状态设为 Dead，年龄归零
- tick(): 存活一代，年龄加 1

2.2.2 Grid ——网格类

Grid 类

Grid 是整个游戏的”世界”，管理所有细胞的位置和状态。支持两种模式：

- 固定模式: 使用 vector<vector<Cell>> 存储，支持死边界和环形边界。死边界下边界外的邻居被视为死细胞；环形边界下边界两侧连通。
- 无限模式: 使用 unordered_set<pair<int,int>> 存储活细胞坐标，内存按需分配，无边限制制。

两个模式共用统一的接口，上层代码无需区分。

2.2.3 Rule ——规则类

Rule 类

Rule 封装了 B/S 规则逻辑。用户通过 `setBirthRule()` 和 `setSurviveRule()` 设置出生和存活条件。提供 `shouldBirth()` 和 `shouldSurvive()` 方法供 Simulator 查询。

支持 `toString()` / `fromString()` 实现规则和字符串（如“B3/S23”）的双向转换，方便在 UI 中展示和解析。

邻居类型通过 `NeighborhoodType` 枚举（Moore / VonNeumann）控制。

2.2.4 Simulator ——演化引擎

Simulator 类

Simulator 继承自 `QObject`，使用 `QTimer` 驱动自动演化。核心方法：

- `step()`：执行单步演化，根据当前 Rule 计算每个细胞的下一状态
- `start(ms)`：启动定时器，以指定间隔自动演化
- `stop()`：暂停演化
- `countNeighbors()`：计算指定细胞在指定邻居类型下的存活邻居数

通过 Qt 信号槽机制通知 UI 更新：`generationChanged()` 和 `statsUpdated()`。

2.2.5 Pattern ——图案类

Pattern 类

Pattern 是一个数据结构，包含图案的名称、描述、分类、适用规则和活细胞坐标列表。提供 `boundingRect()`、`rotated()`、`flipped()` 等工具方法，支持图案的旋转（90°/180°/270°）和翻转操作。

2.3 UI 类设计

2.3.1 MainWindow ——主窗口

MainWindow 是整个程序的顶层容器，包含菜单栏、工具栏、中央区域和状态栏。中央区域水平布局：左侧为可隐藏的 `SideBar`，右侧为 `GridView` 网格视图。它持有 `Grid`、`Simulator` 和 `GridView` 的引用，协调各组件之间的通信。

2.3.2 GridView —— 网格视图

GridView 是核心交互控件，继承自 QWidget。负责：

- 渲染细胞（根据年龄计算渐变色）
- 渲染网格线和图案预览
- 处理鼠标事件（三种模式：Draw / Pan / Select）
- 处理键盘事件（ESC 取消选择）和滚轮缩放
- 提供右键上下文菜单（复制、保存、填满、清空）

2.3.3 SideBar —— 侧边栏

SideBar 是一个 QWidget 容器，内嵌三个面板：RulePanel（规则面板）、PatternPanel（图案面板）、StatsPanel（统计面板）。每个面板以折叠组（QGroupBox）形式呈现，支持展开/折叠。

2.3.4 规则面板（RulePanel）

显示预设规则列表（QComboBox），用户可选择康威、HighLife 等预设规则。提供”自定义”选项，点击后打开 RuleEditDialog 允许用户手动编辑 B/S 规则和邻居类型。

2.3.5 图案面板（PatternPanel）

加载 data/patterns.json 中的预设图案库，按分类（振荡器、飞船、枪等）展示。用户点击图案可预览，再次点击放置到网格。支持加载本地自定义图案。

2.3.6 统计面板（StatsPanel）

实时显示演化统计信息：当前代数、活细胞数量、当前规则、邻居类型。

2.4 对话框类设计

- **GridSettingsDialog**: 设置网格尺寸、边界类型、网格模式（固定/无限）。
- **RuleEditDialog**: 可视化编辑 B/S 规则，用复选框勾选出生和存活条件，选择邻居类型。
- **PatternSaveDialog**: 输入图案名称、描述和分类，预览图案并保存。

2.5 工具类设计

PatternSerializer 提供图案与 JSON 之间的序列化和反序列化功能。使用 Qt 的 QJson-Document 处理 JSON 格式，支持单个图案的保存/加载和图案库的批量管理。

3 小组成员分工

本项目由 3 名成员协作完成，分工如下：

成员	负责模块
万泽奇	Core 层（核心逻辑层） 负责 core/ 目录下所有源文件和头文件的编写和注释： Cell（细胞类）、Grid（网格类）、Rule（规则类） Simulator（演化引擎）、Pattern（图案数据结构） 实现了细胞状态管理、B/S 规则引擎、固定/无限网格模式切换、演化算法（邻居统计、状态更新）等核心功能
赵思涵	UI 层（用户界面层） 负责 ui/ 目录下所有源文件和头文件的编写和注释（不含 dialogs）： GridView（网格视图控件）、PatternPanel（图案面板） RulePanel（规则面板）、SideBar（侧边栏）、StatsPanel（统计面板） 实现了网格渲染（含年龄颜色渐变）、三种交互模式（绘制/拖动/选择）、平移缩放、框选操作、图案预览与放置、右键菜单等丰富交互功能
史云天	对话框 & 工具 & 主框架 负责 ui/dialogs/、utils/、根目录源文件的编写和注释： GridSettingsDialog、RuleEditDialog、PatternSaveDialog（对话框） PatternSerializer（图案序列化工具） mainwindow、main.cpp（主框架集成） 实现了网格参数设置、规则可视化编辑、图案保存/加载、主窗口布局、菜单栏/工具栏集成、各组件信号槽连接等整合工作

表 1: 小组成员分工表

3.1 协作模式

采用模块化开发模式：核心层（万泽奇）提供稳定的数据结构和算法接口，UI 层（赵思涵）基于核心层接口实现可视化，主框架（史云天）负责全局集成和质量把控。模

块间通过明确的方法接口和 Qt 信号槽机制通信，降低了耦合度，三人可以并行开发，显著提高了效率。

4 项目总结与反思

4.1 项目成果

经过团队协作，我们成功实现了一个功能完善的生命游戏 GUI 程序。程序支持：

- 6 条预设规则 + 完全自定义的 B/S 规则编辑
- 2 种邻居类型、2 种边界模式、2 种网格模式
- 内置 20+ 经典预设图案，支持自定义图案保存
- 3 种交互模式（绘制、拖动、选择），流畅的平移缩放体验
- 细胞年龄渐变着色，直观展示演化历史

4.2 技术收获

1. **分层架构的价值：**核心逻辑与界面分离的设计使代码结构清晰，易于维护和测试。核心层的 Cell、Grid、Rule 等类完全不依赖 Qt GUI，可以在命令行环境下独立测试。
2. **Qt 信号槽机制的威力：**Simulator 通过信号通知 UI 更新，使得演化引擎和界面之间的耦合降到最低。这种观察者模式非常适合事件驱动的 GUI 应用。
3. **数据结构选择的重要性：**固定网格使用 `vector<vector>` 访问快但内存固定；无限网格使用 `unordered_set` 内存灵活但访问慢。根据场景选择合适的结构是性能优化的关键。
4. **渲染优化：**GridView 的 `paintEvent` 中使用了 `QPainter` 的变换矩阵实现平移和缩放，避免了手动计算所有坐标。适当关闭抗锯齿提升了方块状细胞的边缘清晰度。

4.3 不足与改进方向

1. **性能瓶颈：**当前演化算法在大型网格下性能有待优化。未来可考虑使用位运算或 GPU 加速（如 OpenCL）来提升计算速度。
2. **无限网格体验：**无限网格模式下，视口以外的演化不可见，用户可能丢失有趣的图案。可考虑添加自动追踪功能。
3. **图案分享：**目前仅支持本地保存图案。添加在线图案分享功能将大大增强社区互动。

4. **多线程演化**: 当前演化在主线程执行, 大网格时可能阻塞 UI。可考虑将计算移至工作线程, 使用双缓冲 Grid 实现无锁更新。
5. **撤销/重做**: 目前不支持撤销操作, 误操作后无法恢复。添加 QUndoStack 将明显提升用户体验。

4.4 结语

通过本项目, 我们不仅深入理解了细胞自动机的工作原理, 也全面实践了 Qt GUI 开发、CMake 构建系统、JSON 序列化、分层架构设计等工程技术。从初始设计到最终实现, 每一步都充满了挑战和收获。生命游戏虽然规则简单, 但其中蕴含的复杂性和美感令人着迷——正如 John Conway 所说, 简单规则可以创造出无限的可能。